Problem 8: KDA recurrence and stable gated scores

GStar Bootcamp

Abstract

Implement recurrent KDA and stable channel-gated scores with secondary tiling.

Starter file: `problem_8.py`. **Checker:** `python autograder.py -p8`. **Reading path:** Overview, P4, P7. Next: P9.

Mathematical background: channel gates change the transition geometry

Scalar forgetting ties every key channel to the same retention rate. KDA instead uses $\mathbf{G}_t = \mathrm{Diag}(e^{\boldsymbol{g}_t})$ and applies the delta correction to the already-decayed state [1]:

$$\mathbf{S}_t = (\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top)\mathbf{G}_t \mathbf{S}_{t-1} + \beta_t \boldsymbol{k}_t \boldsymbol{v}_t^\top.$$

The transition can also be written

$$\mathbf{T}_t = \mathbf{G}_t - \beta_t \boldsymbol{k}_t (\mathbf{G}_t \boldsymbol{k}_t)^\top.$$

It is a diagonal matrix plus a rank-at-most-one modification. This structure supplies channel-specific forgetting while retaining a compact correction. It does not mean that $\mathbf{G}_t$ and the delta transition commute: the key direction need not align with the coordinate axes on which the gate acts.

Transporting an earlier write through channel gates gives an interaction

$$\boldsymbol{x}_r^\top \mathrm{Diag}(e^{\boldsymbol{b}_r - \boldsymbol{b}_j})\boldsymbol{k}_j = \sum_d X_{rd} K_{jd} e^{b_{rd} - b_{jd}}.$$

Each channel can have a different decay, so one scalar multiplier applied after the dot product cannot generally reproduce it. P8's reusable score builder computes this expression for either keys or queries on the left. Strict causality encodes earlier corrections; inclusive causality encodes reads after the current write.

The real-arithmetic identity $e^{b_r - b_j} = e^{b_r - a} e^{a - b_j}$ allows matrix-product operands, but numerical safety depends on choosing an anchor that makes both factors bounded. The subsequent derivation explains why off-diagonal and diagonal secondary tiles need different paths. This numerical reformulation preserves the recurrence; it does not clip valid causal interactions.

Before coding. Explain why broadcasting one scalar gate across all channels recovers P7, and identify the channel axis that must remain inside a gated score reduction.

From scalar to channel-wise forgetting

KDA replaces scalar decay with an independent forget gate for every key channel [1]:

$$\overline{\mathbf{S}}_t = \mathrm{Diag}(e^{\boldsymbol{g}_t})\mathbf{S}_{t-1}, \tag{8.1}$$

$$\boldsymbol{u}_t = \beta_t(\boldsymbol{v}_t - \overline{\mathbf{S}}_t^\top \boldsymbol{k}_t), \tag{8.2}$$

$$\mathbf{S}_t = \overline{\mathbf{S}}_t + \boldsymbol{k}_t \boldsymbol{u}_t^\top, \quad \boldsymbol{o}_t = \mathbf{S}_t^\top \boldsymbol{q}_t. \tag{8.3}$$

Thus

$$\mathbf{S}_t = (\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top)\,\mathrm{Diag}(e^{\boldsymbol{g}_t})\mathbf{S}_{t-1} + \beta_t \boldsymbol{k}_t \boldsymbol{v}_t^\top. \tag{8.4}$$

The order of the rank-one transition and diagonal decay matters. Unlike scalar decay, a diagonal gate generally does not commute with the rank-one transition.

The reusable score primitive

For a left tensor $\mathbf{X}$ and keys $\mathbf{K}$, implement

$$F(\mathbf{X}, \mathbf{K})_{ru} = \sum_{d=1}^{D_k} X_{rd} K_{ud} e^{b_{rd} - b_{ud}}. \tag{8.5}$$

Use a strict mode $u < r$ for the key–key interaction in the delta system, and an inclusive mode $u \le r$ for query–key output scores. Valid entries must match Eq. (8.5); all invalid entries must be zero.

Two uses of the same primitive

Memory correction: $F(\mathbf{K}, \mathbf{K})$, strict mask, scale 1.
Output: $F(\mathbf{Q}, \mathbf{K})$, inclusive mask, with query scale applied exactly once by the output pipeline.

Why an ordinary dot product is insufficient

For a scalar gate, the decay factor is shared across the key dimension and can multiply a completed dot product. For channel gates, the decay depends on d inside the sum. You must incorporate those factors before or during reduction over key channels.

Problem 8: secondary tiling prevents overflow

An unsafe algebraic factorization

In exact arithmetic, $e^{b_{rd} - b_{ud}} = e^{b_{rd}} e^{-b_{ud}}$. In FP32 the individual terms can underflow or overflow. For example,

$$b_u = -100, \quad b_r = -101, \quad e^{b_r - b_u} = e^{-1} \approx 0.3679,$$

while e^{100} exceeds the FP32 finite range. Multiplying separate transformed inputs can produce a NaN even though the valid relative decay is small.

Off-diagonal secondary tiles

Divide each $C = 64$ chunk into $R = 16$ token tiles. For query tile i strictly after key tile j, choose the channel-wise prefix at the *end of key tile* j as anchor $\mathbf{a}$:

$$\widetilde{X}_{rd} = X_{rd} e^{b_{rd} - a_d}, \tag{8.6}$$

$$\widetilde{K}_{ud} = K_{ud} e^{a_d - b_{ud}}, \tag{8.7}$$

$$F(\mathbf{X}, \mathbf{K})_{\text{tile}} = \widetilde{\mathbf{X}} \widetilde{\mathbf{K}}^\top. \tag{8.8}$$

Since log-prefixes are nonincreasing and all key positions precede all query positions, $b_{rd} \le a_d \le b_{ud}$. Both exponent arguments are nonpositive. This supports bounded operands for a matrix product.

Diagonal secondary tiles

Rows in the same secondary tile can overlap in time. No single anchor places both factors at most one for every causal pair. Compute the relative gate difference directly in FP32. Replace invalid pair arguments with a safe value *before* exponentiation, then zero invalid scores. Reduce the channel dimension to obtain an $R \times R$ score tile.

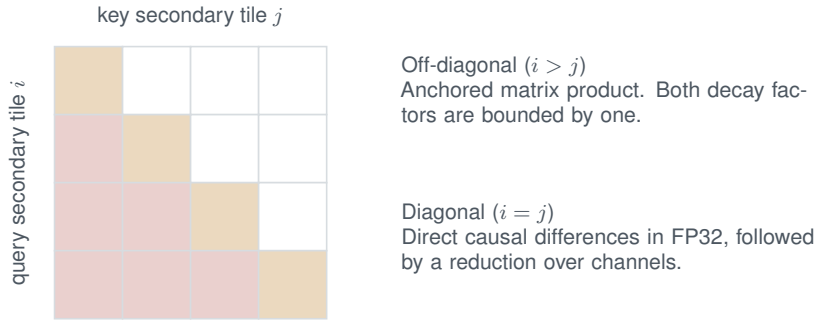


Figure 1. Secondary tiling separates bounded matrix-product tiles from diagonal tiles that require direct evaluation. Each large cell represents a 16×16 token tile.

Problem 8: implementation steps and checks

1. **Recurrent KDA (30 points).** Extend Problem 7's recurrent kernel to a channel gate. Decay state rows, then compute the residual and write. Verify the order in Eq. (8.3).
2. **Stable score builder (50 points).** Complete `_chunk_kernel_matrix`: choose the supplied anchor; transform off-diagonal left/key operands; accumulate their matrix product; compute direct FP32 diagonal tiles; apply the strict or inclusive mask. Complete the small `gated_scores_diagonal` helper as part of this subpart.
3. **Numerical explanation (20 points).** Explain the unsafe e^b/e^b factorization and prove that both anchored off-diagonal exponent arguments are nonpositive when gates satisfy the contract.

```
# Secondary-tile loops, tile loads, and anchor index are provided.
if query_tile == key_tile:
    # TODO P8.A: evaluate safe relative gate differences in FP32.
    # TODO P8.B: reduce over key channels, then apply pair mask.
else:
    # TODO P8.C: transform left and key using the shared anchor.
    # TODO P8.D: accumulate the bounded matrix product.
# TODO P8.E: distinguish strict versus inclusive diagonal mode.
```

Verify the primitive before the full operator

Compare every valid entry with a direct FP64 implementation of Eq. (8.5), and confirm invalid entries are exactly zero. Test $\mathbf{X} = \mathbf{Q}$ and $\mathbf{X} = \mathbf{K}$, both mask modes, two key tiles, and strong unequal channel gates.

Table 1. Verify the primitive before the full operator.

Condition	**Expected result**
Zero gates	Ordinary masked query/key or key/key dot products
Inclusive diagonal	$F(\mathbf{X}, \mathbf{K})_{rr} = \boldsymbol{x}_r^\top \boldsymbol{k}_r$
Strict diagonal	Every diagonal entry equals zero
Scalar gate broadcast	Recurrence matches Gated DeltaNet
Unequal channel gates	Channel decay remains inside the dot-product reduction
Strong negative prefixes	All outputs finite; no separate growing exponent

Use this primitive to replace Problem 4's direct score baseline, and then reuse it in Problem 9. It must not call an upstream FLA operator.

Initialize what later kernels will read

Some score builders store only lower-triangle tiles. The supplied launcher must zero the complete score buffer before those stores, or the kernel must explicitly store zeros in every unused tile. A later full-tile load must never read uninitialized upper-triangle entries.

Problem 8: channel gates, noncommutativity, and stable anchors

A concrete noncommuting transition

Let $\boldsymbol{k} = (1,1)^\top/\sqrt{2}$, $\beta = 1$, and $\mathbf{G} = \mathrm{Diag}(1/4, 1)$. The delta transition is $\mathbf{T} = \mathbf{I} - \boldsymbol{k}\boldsymbol{k}^\top$. Direct multiplication gives

$$\mathbf{TG} = \begin{bmatrix} 1/8 & -1/2 \\ -1/8 & 1/2 \end{bmatrix}, \qquad \mathbf{GT} = \begin{bmatrix} 1/8 & -1/8 \\ -1/2 & 1/2 \end{bmatrix}.$$

The matrices differ. KDA needs $\mathbf{TGS}_{t-1}$, corresponding to forgetting before residual prediction. Moving forgetting after the delta transition implements the other product and changes the operator.

A bounded off-diagonal factorization

Take one channel with key prefix $b_j = -100$, an anchor $a = -104$ at the end of its key subtile, and a later query prefix $b_r = -106$. The exact relative decay is

$$e^{b_r - b_j} = e^{-6} = e^{b_r - a}\, e^{a - b_j} = e^{-2} e^{-4}.$$

Both factors are at most one. By contrast, separate chunk-global transforms would contain $e^{-b_j} = e^{100}$, which overflows FP32. The anchor cancels a large common prefix before exponentiation.

The general bound follows from monotonicity: $b_{rd} \le a_d \le b_{jd}$ for each channel in a strictly off-diagonal causal subtile. Thus $b_{rd} - a_d \le 0$ and $a_d - b_{jd} \le 0$. Apply this proof channel by channel, not only to the average prefix.

Why diagonal subtiles require direct differences

For two positions inside the same subtile, a query may be earlier than the chosen end-of-key-subtile anchor. Then $b_{rd} - a_d$ is positive, and one of the separately transformed factors can grow. For these subtiles compute the full relative difference for each token pair in FP32, apply a safe exponent argument, and reduce over key channels.

The direct helper temporarily forms an $R \times R \times B_K$ tensor, not a full $C \times C \times D_k$ tensor. With $R = 16$ and $B_K = 64$, this is 16,384 scalar elements before compiler lowering. Secondary tiling confines this work to diagonal subtiles while off-diagonal subtiles use ordinary matrix products.

The mask and the scale are separate arguments

For strict mode, all $r = j$ entries are zero. For inclusive mode, the diagonal is $\boldsymbol{x}_r^\top \boldsymbol{k}_r$, since relative decay is one. The key–key WY call always uses scale 1; the query–key output call uses the output scale once. Initialize the complete score buffer to zero so upper subtiles that are never visited cannot contain arbitrary values.

References

[1] Kimi Team. *Kimi Linear: An Expressive, Efficient Attention Architecture*. 2025. https://arxiv.org/abs/2510.26692